

CEG 4350/5350 – Operating Systems Internals and Design

Project 1 – Hello World in Your OS

Learning Objectives

- Learn the basics of writing a simple operating system and interface with hardware.

Overview

In this project you will become familiar with writing low level OS code, bootloaders, and printing to the display.

Contents of New Project

Once the setup procedure is complete, you can start writing custom code for your OS.

Look through the project files and you will notice 4 folders, these are their uses:

- asm
 - NASM assembly language files (.asm)
 - Contains the bootloader that sets up your system and loads your kernel
- build
 - All compiled object and binary files (.o, .bin, .img)
 - When you write new .c files and you compile your OS, their object files will appear here
- include
 - C header files (.h)
 - When you write new .c files, create a new .h file for function declarations and macro definitions (see example)
- src
 - C source files (.c)
 - This is the bulk of your operating system, you will write C code .c files here to expand the functionality of your OS

Hello World

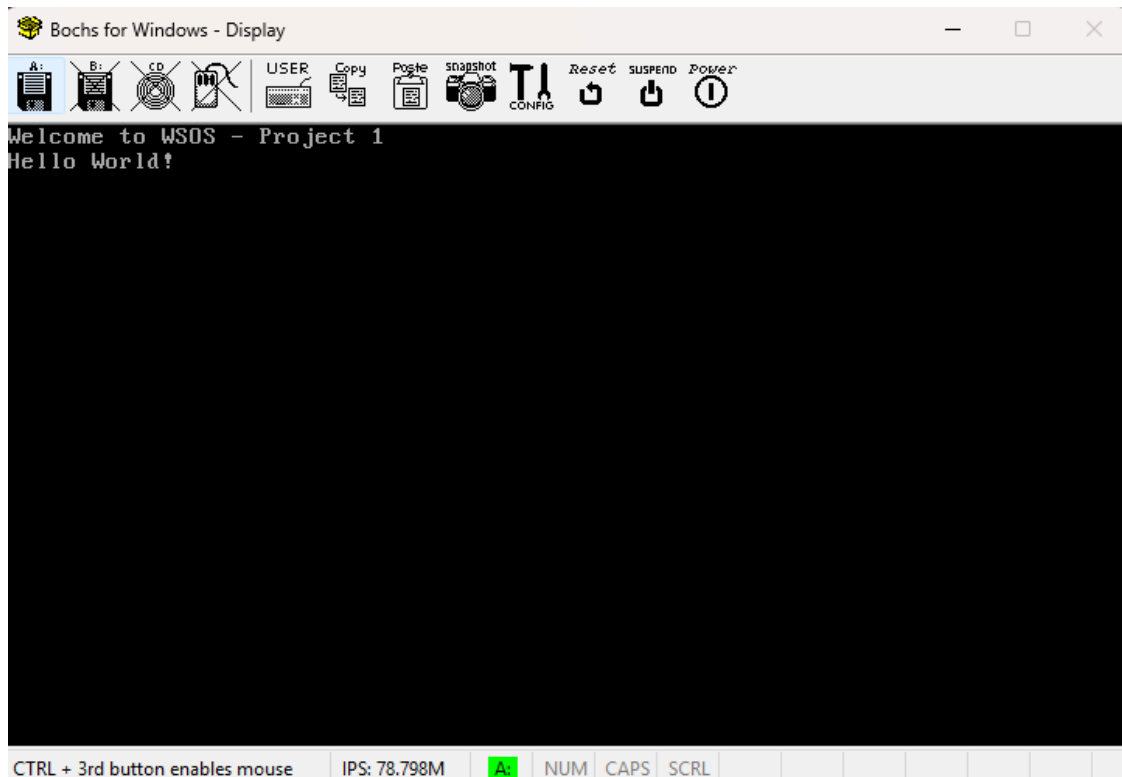
For this first project you have 3 milestones that will allow you to print “Hello World!” to the display. After you have made changes to your code and you want to test your code run “wsl make PROJECT=1” in the terminal in VS Code, then add the image to Bochs and start your OS.

1. Modify project1.c and project1.h to implement 5 functions:
 - a. void setcursor(int x, int y) must set the current cursor position using the global variables cursorCol and cursorRow. There will not be any visual indication that anything has changed but print and putchar should now use cursorCol and cursorRow to print at this location on the display. If int x exceeds the screen’s width, continue to the next row and wrap the column. This can be easily accomplished using the % operator. If int y exceeds the screen’s height, call the scroll() function from Project 2 (not implemented yet) to scroll the difference of rows, i.e. cursorRow - SCREEN_HEIGHT + 1 and set cursorRow to SCREEN_HEIGHT – 1. The cursor position variables cursorCol and cursorRow should always be in the range of (0 to 79) and (0 to 24), respectively.
 - i. int x - the column (horizontal) position on the display to set the cursor
 - ii. int y - the row (vertical) position on the display to set the cursor to
 - iii. returns nothing
 - b. char putchar(char character) must print the character to the display wherever specified by int x and int y is pointing at using color #define TEXT_COLOR 0x07. Then, call setcursor() to increment the column by 1. If a newline character ('\n') is passed as an input, the cursor should move to the next row, column 0.
 - i. char character - the character to be printed to the display
 - ii. returns the character printed
 - c. int print(char string[]) must print the string to the display wherever the current cursor is pointing at. After printing the string the cursor must be set to the end of the string.
 - i. char string[] - the string to be printed to the display
 - ii. returns the number of characters printed to the display
 - d. void clearscreen() must remove all text from the screen and reset cursor position to 0, 0. This can be accomplished by overwriting all video memory locations with a space ‘ ‘ character.
 - e. int kernel() must call the print function that you wrote to print “Hello World!” to the display.
 - i. returns 0

Hint: You can reuse your functions inside of your other functions. For example, instead of writing putchar() and print() separately, simply let print() call putchar() multiple times to print the string. Also inside of putchar() you can increment the cursor position with setcursor().

Hint: When writing your own OS, you do not have access to any standard libraries (i.e. stdio.h for print()). For this reason, we must write print() from scratch. Do not waste time trying to use standard C libraries. Trying to use them will leave you frustrated because they will not work. If you need something for your code, you probably have to write it from scratch.

If your OS implements what is described above you will see this result:



How to Submit

To submit your code, commit your changes to your GitHub repository and include a link to your repository in the Pilot dropbox.

Grading

This lab is worth 5.00 points, distributed as follows:

Task	Points
OS compiles without errors (warnings are ok)	0.50
OS boots without errors (warnings are ok)	0.50
setcursor() is implemented	1.00
putchar() is implemented	1.00
print() is implemented	1.00
clearscreen() is implemented	1.00
Total	5.00